

Docs

Spield — Yield stripping finance on Stellar network

Tokenize yield on Stellar by splitting stsUSDe into principal (PT) and yield (YT).

Spield is a yield-stripping protocol on Stellar. It turns **one yield-bearing token** into two tradable positions:

- **Principal Token (PT)**: fixed outcome at maturity.
- **Yield Token (YT)**: variable outcome, linked to realized yield.

Spield does not remove yield risk. It **routes yield risk** to the users who want it.

What you can do

- Lock predictable value via PT (bond-like exposure).
- Speculate on yield via YT (leveraged exposure).
- Trade PT and YT independently before maturity.

Doc set

Read in this order:

1. [Product overview](#)
2. [Yield stripping model \(PT + YT\)](#)
3. [User flows](#)
4. [Yield index and yield calculation \(YT\)](#)
5. [Maturity, settlement, and capital conservation](#)
6. [Orderbook marketplace \(PT/YT trading\)](#)
7. [MVP assumptions and parameters](#)

① Key formulas used in these docs:

- **YT payout** = $(\text{currentIndex} - \text{pastIndex}) \times \text{total YT bought}$
- **APY** = $((\text{currentIndex} / \text{sevenDaysOldIndex})^{(365/7)}) - 1$

Product overview

Spield tokenizes yield for **stsUSDe** on Stellar. It splits each yield-bearing unit into **principal** and **yield**.

Terminology

- **stsUSDe**: the yield-bearing token deposited into Spield.

Core idea

- You start with **1 stsUSDe** (a yield-bearing token).
- Spield mints two positions against it:
 - **1 PT-stsUSDe** (principal claim)
 - **1 YT-stsUSDe** (yield claim)

Before maturity, PT and YT are tradable. At maturity, they settle back into economic reality.

Why this exists

Yield is uncertain. Most DeFi forces everyone to hold the same uncertainty.

Spield makes yield risk a choice:

- **Conservative users** buy PT for predictable outcomes.
- **Risk-tolerant users** buy YT for variable outcomes.

Accounting identity (always)

① **$\text{Value}(\text{stsUSDe}) = \text{Value}(\text{PT-stsUSDe}) + \text{Value}(\text{YT-stsUSDe})$**

Example pricing (illustrative):

- $\text{Value}(\text{stsUSDe}) = \mathbf{1.00 \text{ USDe}}$
- $\text{Value}(\text{PT}) = \mathbf{0.95 \text{ USDe}}$
- $\text{Value}(\text{YT}) = \mathbf{0.05 \text{ USDe}}$

So: **$1.00 = 0.95 + 0.05$**

What Spield is not

- Not “risk-free yield”.
- Not a yield guarantee.
- Not capital creation.

Spield only **redistributes** yield outcomes between PT and YT holders.

Architecture

Table of Contents

1. [What is Spield? ↗](#)
2. [High-Level Architecture ↗](#)
3. [Stellar Integration — Deep Dive ↗](#)
 - [Why Stellar Soroban? ↗](#)
 - [Soroban Smart Contracts ↗](#)
 - [Bridge Oracle Contract ↗](#)
 - [Wrapper Contract \(Derivative Engine\) ↗](#)
 - [Marketplace Orderbook Contract ↗](#)
 - [Stellar Asset Contract \(SAC\) ↗](#)
 - [Cross-Contract Calls on Soroban ↗](#)
 - [Soroban Storage Model ↗](#)
 - [Authorization Model ↗](#)
 - [Event System ↗](#)
 - [Freighter Wallet Integration ↗](#)
 - [Transaction Lifecycle on Stellar ↗](#)
4. [Cross-Chain Bridge — EVM ↔ Stellar ↗](#)
 - [Deposit Flow ↗](#)
 - [Withdrawal Flow ↗](#)
 - [Security Model ↗](#)
5. [Yield Oracle ↗](#)
6. [Yield Mechanics ↗](#)
7. [Contract Addresses ↗](#)
8. [Tech Stack ↗](#)
9. [Project Structure ↗](#)
10. [Security Considerations ↗](#)

What is Spield?

Spield allows users to separate yield-bearing assets into two derivative tokens:

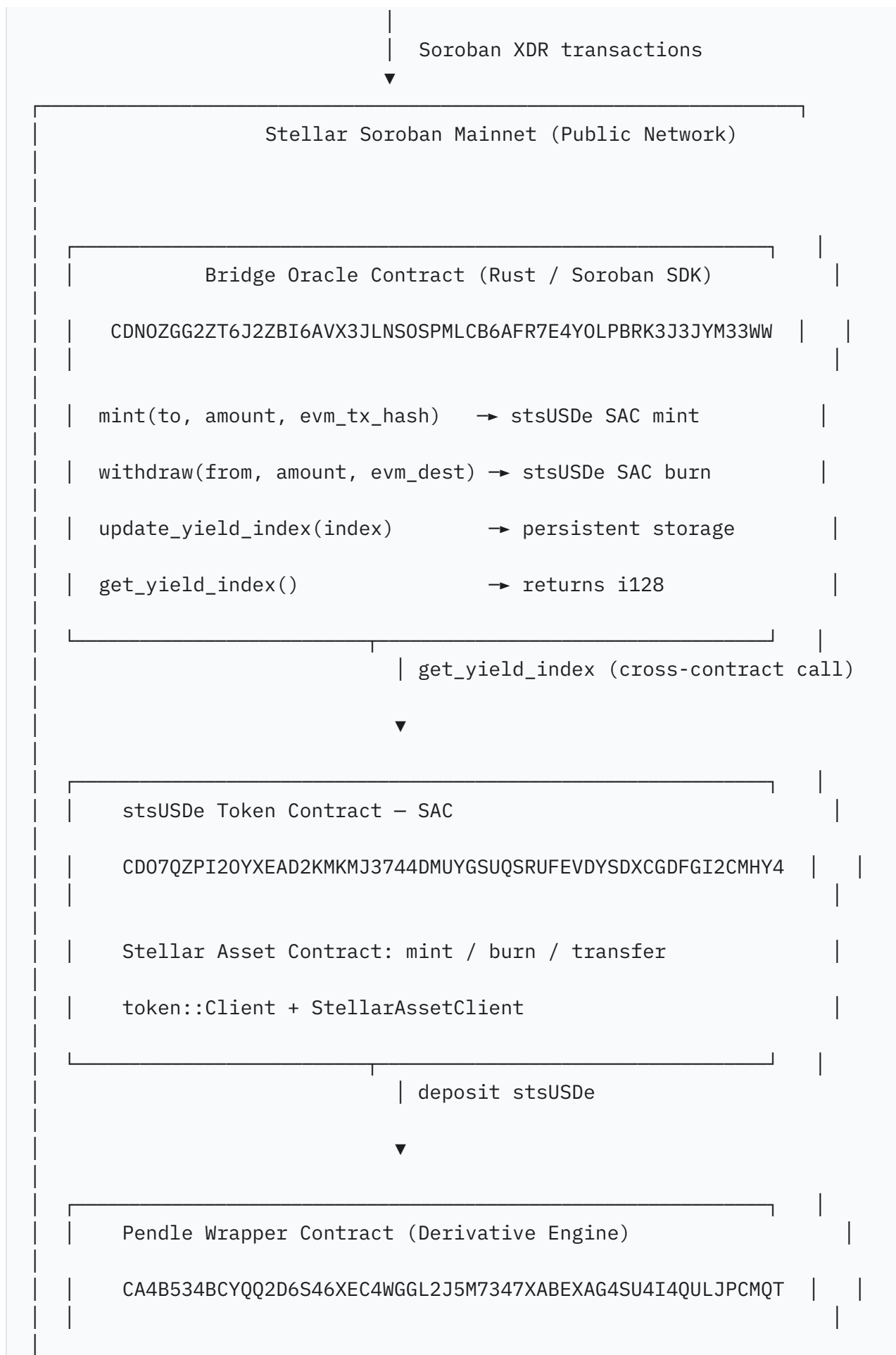
- **PT (Principal Token):** Represents your locked principal. Redeemable 1:1 for the underlying asset at maturity. Trades at a discount before maturity (~0.97 USDC).
- **YT (Yield Token):** Represents accumulated yield since your entry point. Can be claimed at any time by burning all YT and receiving the yield in stsUSDe.

This enables three primary DeFi strategies:

Strategy	Mechanism
Fixed Yield	Buy PT at a discount → redeem at par at maturity
Yield Speculation	Hold YT and bet on yield rate increases
Yield Trading	Sell YT immediately to lock in current yield value

High-Level Architecture





<code>mint_split(user, amount)</code>	→ issues PT + YT 1:1
<code>redeem_pt(user, amount)</code>	→ post-maturity principal back
<code>claim_yield(user)</code>	→ burns YT, pays stsUSDe yield
<code>combine_and_redeem(user)</code>	→ early exit (burns PT + YT)
<code>transfer_pt / transfer_yt</code>	→ Marketplace escrow moves

| list / buy PT and YT



Marketplace Orderbook Contract	
CDHHGWGBGT6APB3CDODQJPGUXLYPIU34VV6PBT7SRRHBY3KEWYJVOQHS	
<code>list_pt / list_yt</code>	→ escrow tokens from Wrapper
<code>buy_market_pt(buyer, amt)</code>	→ auto-match fills, USDC pay
<code>buy_market_yt(buyer, amt)</code>	→ iterate SellersListYT

Frontend: React/Vite + Freightier Wallet + @stellar/stellar-sdk

Stellar Integration — Deep Dive

This section documents every layer of Spield's integration with the Stellar network and the Soroban smart contract platform.

Why Stellar Soroban?

Spield chose Stellar Soroban for the following reasons:

Reason	Detail
Low transaction fees	Sub-cent fees make micro-yield claims economically viable
Fast finality	~5 second ledger close vs. minutes on Ethereum
Rust-native contracts	Type safety and WASM execution for complex yield math
Stellar Asset Contract (SAC)	Native token standard that integrates with Stellar's existing asset framework
Soroban auth model	<code>require_auth()</code> enforces per-invocation signatures without session state
Built-in storage TTLs	Automatic state expiry management for scalable persistent data

Soroban Smart Contracts

All four core contracts are written in **Rust** targeting `wasm32v1-none` and compiled using the **Soroban SDK**. Each contract is independently deployed, upgradeable, and communicates via Soroban's synchronous cross-contract call mechanism.

```
contract_2/  
├─ bridgeContract/      ← Bridge Oracle + stsUSDe SAC admin  
├─ pendleWrapper/      ← PT/YT derivative engine  
└─ orderBook/          ← P2P marketplace with auto-matching
```

All contracts use `#![no_std]` for minimal WASM binary size and deterministic execution.

Bridge Oracle Contract

Address (Mainnet):

CDNOZGG2ZT6J2ZBI6AVX3JLNS0SPMLCB6AFR7E4Y0LPBRK3J3JYM33WW

This is the entry point for all cross-chain operations. It serves two roles: the **EVM bridge** and the **yield oracle**.

State Layout

```
// Instance storage (ephemeral globals, cheap reads)
Relayer          → Address          // authorized relayer
StellarAsset     → Address          // stsUSDe SAC contract address
YieldIndex       → i128             // latest sUSDe index (7-decimal
fixed-point)

// Persistent storage (per-transaction, survives TTL extension)
Processed(BytesN<32>) → bool         // replay protection per EVM tx
hash
```

Key Functions

```
pub fn mint(env: Env, to: Address, amount: i128, evm_tx_hash:
BytesN<32>)
```

- Requires `relayer.require_auth()`
- Checks `Processed(evm_tx_hash)` → panics on duplicate
- Calls `StellarAssetClient::mint(to, amount)` on the SAC
- Extends TTL of `Processed` entry

```
pub fn withdraw(env: Env, from: Address, amount: i128, evm_dest:
String)
```

- Requires `from.require_auth()`
- Burns `amount` from the caller's stsUSDe balance via `StellarAssetClient::burn`

- Publishes a `withdraw` Soroban event (relayer polls these)

```
pub fn update_yield_index(env: Env, index: i128)
```

- Requires `relayer.require_auth()`
- Stores `index` in instance storage under `YieldIndex`
- Extends instance TTL to ensure availability for Wrapper calls

```
pub fn get_yield_index(env: Env) -> i128
```

- Public read — called by the Pendle Wrapper via cross-contract invocation
- Returns current 7-decimal fixed-point index

Wrapper Contract (Derivative Engine)

Address (Mainnet):

```
CA4B534BCYQQ2D6S46XEC4WGGL2J5M7347XABEXAG4SU4I4QULJPCMQT
```

The Wrapper is the core DeFi logic. It manages the lifecycle of PT and YT tokens from minting through maturity.

State Layout

```
// Persistent storage – per-user state survives ledger TTL
UserPT(Address)      → i128    // user's PT balance
UserYT(Address)      → i128    // user's YT balance
UserEntryIndex(Address) → i128  // yield index at time of deposit

// Instance storage – global counters
TotalPT      → i128
TotalYT      → i128
OracleAddr   → Address        // Bridge Oracle for cross-contract
get_yield_index()
Maturity     → u64            // Unix timestamp of maturity
StsusdcAddr  → Address        // stsUSDe SAC address
```

mint_split — The Core Entry Point

```

User calls mint_split(user, amount)
|
|— require user.require_auth()
|— transfer `amount` stsUSDe from user → Wrapper contract (escrow)
|— cross-contract call: BridgeOracle.get_yield_index() →
entry_index
|— set UserEntryIndex(user) = entry_index
|— set UserPT(user) += amount
|— set UserYT(user) += amount
|— update TotalPT, TotalYT

```

claim_yield — Yield Harvesting

```

User calls claim_yield(user)
|
|— cross-contract call: BridgeOracle.get_yield_index() →
current_index
|— read UserEntryIndex(user) → entry_index
|— read UserYT(user) → yt_balance
|
|— profit = (yt_balance × (current_index - entry_index)) /
INDEX_SCALAR
|— INDEX_SCALAR = 10_000_000
|
|— panic if profit <= 0 (no yield has accrued)
|— burn all UserYT(user) = 0
|— transfer `profit` stsUSDe from Wrapper → user
|— publish yield_pay event

```

redeem_pt — Post-Maturity Principal

```

User calls redeem_pt(user, amount_pt)
|
|— panic if ledger.timestamp() < maturity (on-chain gated)
|— burn amount_pt from UserPT(user)
|— transfer amount_pt stsUSDe from Wrapper → user (1:1 recovery)

```

combine_and_redeem — Early Exit

```
User calls combine_and_redeem(user, amount)
|
├─ require UserPT(user) >= amount AND UserYT(user) >= amount
├─ burn amount PT + amount YT simultaneously
└─ transfer amount stsUSDe from Wrapper → user (no yield
captured)
```

Marketplace Orderbook Contract

Address (Mainnet):

```
CDHHGWGBGT6APB3CDODQJPGUXLYPIU34VV6PBT7SRRHBY3KEWYJVOQHS
```

A non-custodial, peer-to-peer orderbook with automatic best-price fill matching. Settlement currency is USDC.

Pricing

```
PT price = 0.97 USDC per unit
YT price = 0.03 USDC per unit
PT + YT = 1.00 USDC ← no-arbitrage parity with underlying stsUSDe
```

State Layout

```
// Persistent storage – per-seller listings
SellOrderPT(Address) → i128 // PT listed for sale
SellOrderYT(Address) → i128 // YT listed for sale

// Instance storage – global seller lists for iteration
SellersListPT → Vec<Address>
SellersListYT → Vec<Address>
```

list_pt / list_yt — Seller Flow

```

Seller calls list_pt(seller, amount)
|
|— require seller.require_auth()
|— cross-contract call: Wrapper.transfer_pt(seller, Marketplace,
amount)
|   → moves PT from seller's balance into Marketplace escrow
|— set SellOrderPT(seller) += amount
|— append seller to SellersListPT (if not already present)

```

`buy_market_pt` — Buyer Auto-Match Fill

```

Buyer calls buy_market_pt(buyer, amount_needed)
|
|— require buyer.require_auth()
|— iterate SellersListPT in order:
|   for each seller:
|       take_amount = min(SellOrderPT[seller], remaining_needed)
|       cost = (take_amount × 97) / 100 ← 0.97 USDC/PT
|       transfer cost USDC: buyer → seller (direct, no fee)
|       cross-contract: Wrapper.transfer_pt(Marketplace, buyer,
take_amount)
|       SellOrderPT[seller] -= take_amount
|       remaining_needed -= take_amount
|       if remaining_needed == 0: break
|— panic if market depth insufficient: "Not enough PT available!"

```

Stellar Asset Contract (SAC)

`stsUSDe` SAC Address:

```
CD07QZPI20YXEAD2KMKMJ3744DMUYGSUQSRUFEVDYSDXCGDFGI2CMHY4
```

The `stsUSDe` token is issued as a **Stellar Asset Contract (SAC)**, which gives it two key properties:

1. It can be held in native Stellar accounts and transacted via Stellar's base layer.
2. The Bridge Contract holds admin rights over the SAC and can call `mint` and `burn` — the two operations required for the cross-chain peg.


```
// How the Bridge Contract interacts with the SAC
use soroban_sdk::token::{StellarAssetClient, Client as TokenClient};

// Mint on deposit
StellarAssetClient::new(&env, &stellar_asset_address).mint(&to,
&amount);

// Burn on withdrawal
StellarAssetClient::new(&env, &stellar_asset_address).burn(&from,
&amount);

// Transfer (used by Wrapper and Marketplace)
TokenClient::new(&env, &stsusdc_address).transfer(&from, &to, &amount);
```

Cross-Contract Calls on Soroban

Spield makes extensive use of Soroban's synchronous cross-contract invocation system. This is how the three contracts communicate:

```
Wrapper → Bridge Oracle
  get_yield_index()
  Called during: mint_split, claim_yield

Marketplace → Wrapper
  transfer_pt(from, to, amount)
  transfer_yt(from, to, amount)
  Called during: list_pt, list_yt, buy_market_pt, buy_market_yt
```

Cross-contract calls in Soroban are deterministic and stack-based — no re-entrancy is possible by design. Each call increments the call stack depth counter and executes synchronously within the same ledger close.

```
// Example: Wrapper calling Bridge Oracle for yield index
let oracle_client = BridgeOracleClient::new(&env, &oracle_address);
let current_index: i128 = oracle_client.get_yield_index();
```

Soroban Storage Model

Spield carefully separates data across Soroban's two storage tiers to optimize for cost and TTL:

Storage Type	Used For	TTL Behavior
Instance	Global state (relayer, asset addresses, yield index, seller lists)	Extended per-contract invocation; cheap access
Persistent	Per-user data (PT/YT balances, entry index, processed hashes, sell orders)	Extended per-key access; survives contract upgrades

```
// Instance storage example (Bridge Oracle – yield index)
env.storage().instance().set(&Symbol::new(&env, "YieldIndex"), &index);

// Persistent storage example (Bridge Oracle – replay protection)
env.storage().persistent().set(&DataKey::Processed(evm_tx_hash.clone()),
    &true);
env.storage().persistent().extend_ttl(&DataKey::Processed(evm_tx_hash),
    0, 100_000);
```

TTL extensions are called explicitly after every write to ensure data does not expire between user interactions.

Soroban Authorization Model

Every state-mutating function in Spield requires an explicit `require_auth()` call. This is Soroban's primitive for signature verification — there is no implicit transaction sender concept (unlike EVM's `msg.sender`).

```
pub fn mint_split(env: Env, user: Address, amount: i128) {
    user.require_auth(); // Freighter signs this invocation
    // ...
}

pub fn mint(env: Env, to: Address, amount: i128, evm_tx_hash:
BytesN<32>) {
    relayer.require_auth(); // Only the authorized relayer keypair
can call
    // ...
}
```

The authorization is embedded in the Soroban XDR transaction built by the frontend and signed by the Freighter browser extension. The contract validates the signature against the stored `relayer` address on-chain.

Soroban Event System

Spield emits structured Soroban events for key protocol actions. These are indexed and queryable via `sorobanServer.getEvents()`.

```
// Withdrawal event (Bridge Contract) – relayer polls this
env.events().publish(
    (Symbol::new(&env, "withdraw"),),
    (from.clone(), amount, evm_dest.clone())
);

// Yield payment event (Pendle Wrapper)
env.events().publish(
    (Symbol::new(&env, "yield_pay"),),
    (user.clone(), profit)
);
```

The Node.js relayer uses a 5-second polling loop with an `isPolling` guard and an in-memory `seenHashes` set to ensure each withdrawal event is processed exactly once:

```
sorobanServer.getEvents({
  startLedger: lastCheckedLedger,
  filters: [{ contractIds: [BRIDGE_CONTRACT_ID], topics: [["withdraw"]]
}]
});
```

Freighter Wallet Integration

The frontend uses the **Freighter browser extension** as the signing backend. The integration supports both Freighter API v5 and v6 for backward compatibility.

Transaction Lifecycle

1. Frontend builds Soroban operation (XDR)
↓
2. `simulateTransaction()` – RPC call to estimate fees + generate footprint
↓
3. `prepareTransaction()` – attaches auth entries and resource fee
↓
4. `signTransaction(xdr, { networkPassphrase })` – Freighter popup
↓
5. `sendTransaction(signedXdr)` – broadcast to Stellar network
↓
6. `pollTransaction(txHash)` – poll until status: "SUCCESS" or "FAILED"

```
// stellar-wrapper.js – simplified write transaction flow
import { signTransaction } from "@stellar/freighter-api";
import { SorobanRpc, TransactionBuilder } from "@stellar/stellar-sdk";

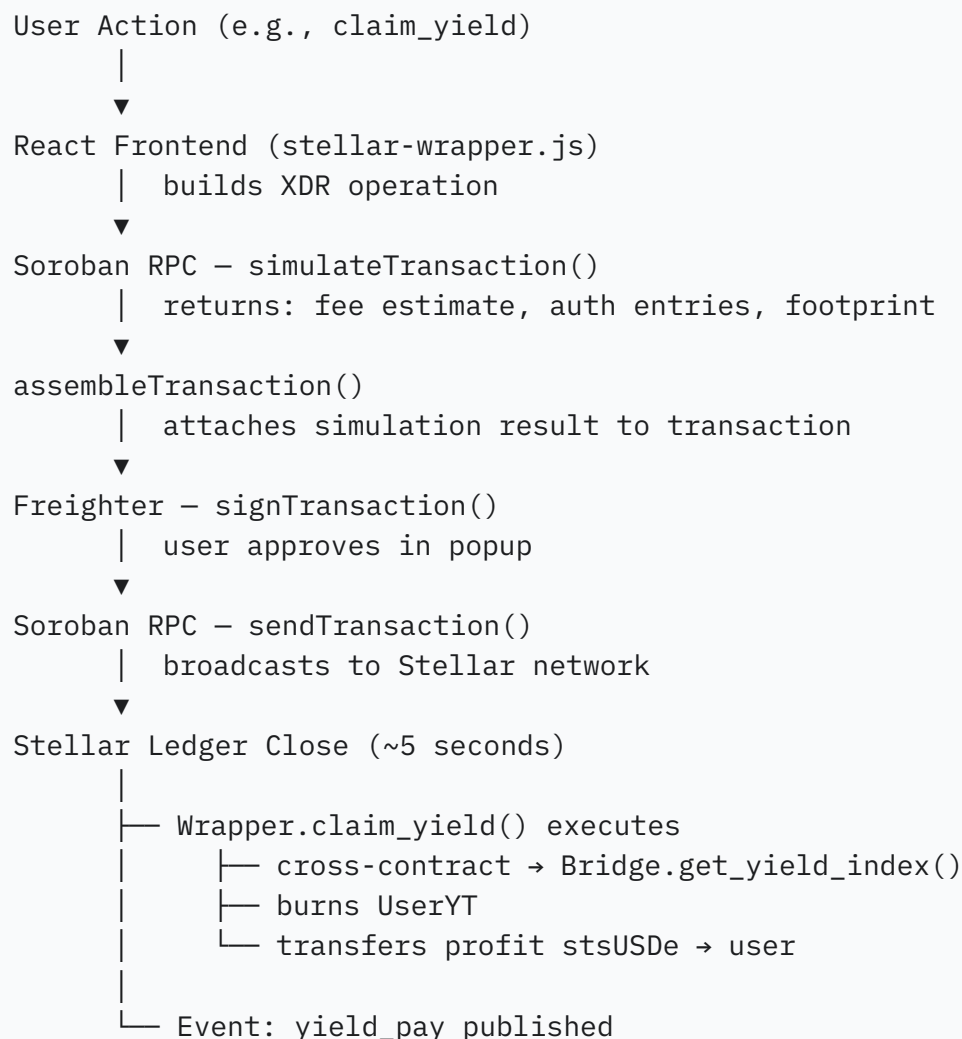
const server = new SorobanRpc.Server(SOROBAN_RPC_URL);
const account = await server.getAccount(userAddress);

const tx = new TransactionBuilder(account, { fee: BASE_FEE })
  .addOperation(contract.call("mint_split", xdrUser, xdrAmount))
  .setTimeout(30)
  .build();

const simulated = await server.simulateTransaction(tx);
const prepared = SorobanRpc.assembleTransaction(tx, simulated).build();
const signed = await signTransaction(prepared.toXDR(), {
  networkPassphrase: Networks.PUBLIC });
const result = await
  server.sendTransaction(TransactionBuilder.fromXDR(signed,
    Networks.PUBLIC));
```

Read-only calls use `simulateTransaction` against a static demo address — no wallet connection required for viewing balances and market data.

Transaction Lifecycle on Stellar



Cross-Chain Bridge — EVM ↔ Stellar

Deposit Flow (EVM → Stellar)

1. User calls EVM Vault `lockTokens(stellarAddress, amount)` on Sepolia
| ERC-20 USDC transferred to Vault custody
2. Vault emits `TokensLocked(evmSender, stellarDest, amount, nonce)`
|
3. Node.js Relay detects event (ethers.js event listener)
| Waits 2 block confirmations for finality

- | Converts: $\text{amount} \times 10$ (18-decimal EVM $\rightarrow$ 7-decimal Soroban)
- 4. Relayer calls `Bridge.mint(stellarAddress, convertedAmount, evmTxHash)`
 - | Requires relayer keypair signature
- 5. Bridge Contract on Soroban:
 - └ Checks `Processed(evmTxHash) = false` (replay guard)
 - └ Calls `SAC.mint(stellarAddress, amount)`
 - └ Sets `Processed(evmTxHash) = true`
- 6. User receives `stsUSDe` tokens in their Stellar wallet

Withdrawal Flow (Stellar $\rightarrow$ EVM)

1. User calls `Bridge.withdraw(user, amount, evmDestAddress)` on Soroban
 - | Requires `user.require_auth()` (Freighter signature)
2. Bridge Contract:
 - └ Calls `SAC.burn(user, amount)` – destroys `stsUSDe`
 - └ Publishes `withdraw` event (`evmDest, amount`)
3. `Node.js` Relayer polls Soroban events every 5 seconds
 - | `isPolling` guard prevents concurrent processing
 - | `seenHashes` set deduplicates within session
 - | Converts: $\text{amount} \div 10$ (7-decimal Soroban $\rightarrow$ 6-decimal EVM)
4. Relayer calls `EVM Vault.unlockTokens(evmAddress, amount, sorobanTxHash)`
 - |
5. Vault releases native USDC to the user's EVM address

Security Model

Threat	Mitigation
Replay attacks	<code>Processed(BytesN<32>)</code> stored persistently; panics on duplicate mint
Unauthorized minting	<code>relayer.require_auth()</code> — only the relayer keypair can call <code>mint</code>
Double withdrawal processing	<code>isPolling</code> guard + <code>seenHashes</code> in-memory set in relayer
Maturity bypass	<code>ledger.timestamp() < maturity</code> enforced entirely on-chain
Yield index manipulation	<code>update_yield_index</code> requires <code>relayer.require_auth()</code>
Re-entrancy	Impossible by Soroban design — synchronous, stack-based execution

Yield Oracle

The Yield Oracle connects live Ethereum Mainnet yield data to Soroban's on-chain state.

Flow

```

Every 1 hour:
  |
  |— Relayer calls sUSDe.convertToAssets(1e18) on Ethereum Mainnet
      Contract: 0x83f20f44975d03b1b09e64809b757c47f942beea (ERC-
4626)
      |
      |— Returns: how many underlying assets 1 share equals (18-
decimal)
      |
      |— Convert to 7-decimal: raw_index / 100_000_000_000
      |
      |— Relayer calls Bridge.update_yield_index(converted_index)
          | Requires relayer.require_auth()
          |— Stores in instance storage → available for Wrapper
calls
  
```

Fixed-Point Representation

```

INDEX_SCALAR = 10_000_000    (represents 1.00000000)

10_000_000 → 1.00000000  (protocol inception)
10_500_000 → 1.05000000  (5% cumulative yield)
11_234_877 → 1.1234877   (~12.35% yield, example live value)
  
```

Yield Mechanics

Profit Formula

$$\text{profit} = \frac{\text{YT Balance} \times (\text{current_index} - \text{entry_index})}{10,000,000}$$

Example Walkthrough

User deposits 100 stsUSDe at yield index 10_000_000

→ receives 100 PT + 100 YT

→ UserEntryIndex = 10_000_000

Oracle pushes new index: 10_500_000 (5% gain)

User calls claim_yield():

profit = $(100 \times (10_500_000 - 10_000_000)) / 10_000_000$

= $(100 \times 500_000) / 10_000_000$

= 5 stsUSDe

→ 100 YT burned

→ 5 stsUSDe transferred to user

→ 100 PT still held (redeemable at maturity for 100 stsUSDe)

Token Value Summary

Token	Value at Maturity	Value Before Maturity	Market Price
PT	1:1 with principal	Time-discounted	~0.97 USDC
YT	No principal value	Speculative yield claim	~0.03 USDC

Contract Addresses

Mainnet (Stellar Public Network)

Contract	Address
Bridge Contract / Yield Oracle	CDNOZGG2ZT6J2ZBI6AVX3JLNSOSPMLCB6AFR7E4 YOLPBRK3J3JYM33WW

stsUSDe Token Contract (SAC)	CD07QZPI20YXEAD2KMKMJ3744DMUYGSUQSRUFEV DYS DXCGDFGI2CMHY4
OrderBook Contract	CDHHGWGBGT6APB3CDODQJPGUXLYPIU34VV6PBT7 SRRHBY3KEWYJVOQHS
Wrapper Contract	CA4B534BCYQQ2D6S46XEC4WGGL2J5M7347XABEX AG4SU4I4QULJPCMQT

Testnet (Stellar Testnet)

Contract	Address
Bridge Contract / Yield Oracle	CDN70WS4YXICY3PULMLRFDC3URCAW233FMXXMSR CEIOV7WYP7SROK3UZ
Wrapper	CDTEVYD4LOV24UMVJNFMM2K7PKBPDPRTEWMLBGA FUXICD6UDFB4M4IFR
Marketplace / OrderBook	CBGKPNLESKNVWRHRAAOPABPZ2QLENAYKBUFOYJK UBDNIEKNAEZG5RHW6

EVM (Sepolia Testnet)

Contract	Address
EVM Vault (deposit/withdraw)	0x0F1cE319fA97F2d8f06661C86696C5417601E F13
sUSDe (yield source, mainnet)	0x83f20f44975d03b1b09e64809b757c47f942b eea

Verified Stellar Testnet Wallets

#	Address
1	GAJLQCVFZNVFB0WVXXWGXB44FJX72CKAB45XAN YPERPIKI5V5MXGWP5
2	GA43F0SDGG7Z7QUT5YVDXFGGPQKWMF72H0HID6S QRXSEUQ00SYBWIRUU
3	GCJWNUQIGTRVWB5Z35JKK55QGNC6ETC22SMYE5H KNNPVQJSGU5HL427N
4	GDG6LWK5R5CKD3M2KJ765MUYXIQHVCQFS32S3BP OYNF4YS6E2W4J4B3F
5	GBKM52FA6XTDQL775MG5KWLAGU3MPHKGU2VRXWZ R2FE773YBBULBFUZV
6	GAWLAMVZVZLBCSAIQYJULRAAJW3G5W4GPUJZF3Q BKSXKEM4TRED27JOE

Tech Stack

Smart Contracts

Layer	Technology
Language	Rust (<code>#![no_std]</code>)
Framework	Soroban SDK
Compilation Target	<code>wasm32v1-none</code>
Storage	Instance (ephemeral globals) + Persistent (per-user state)
Authorization	<code>require_auth()</code> on all state-mutating calls

Token Standard	Stellar Asset Contract (SAC) via <code>token::Client</code> and <code>StellarAssetClient</code>
Cross-contract Calls	<code>env.invoke_contract()</code> for yield index fetch
Events	<code>env.events().publish()</code> for bridge and yield payout tracking

Relayer (Node.js)

Component	Details
Runtime	Node.js >= 18
EVM Connectivity	<code>ethers.js</code> v6
Soroban Connectivity	<code>@stellar/stellar-sdk</code>
Bridge listener	Event-driven via <code>vaultContract.on("TokensLocked", ...)</code>
Withdrawal watcher	Polling loop every 5 seconds via <code>sorobanServer.getEvents()</code>
Yield Oracle	Polling loop every 1 hour via <code>setInterval</code>
Security	<code>dotenv</code> for secrets; in-memory <code>seenHashes</code> for deduplication

Frontend (React/Vite)

Component	Details
Framework	React 18 + Vite
Wallet	Freighter API (<code>@stellar/freighter-api</code>) — supports v5 & v6
Chain SDK	<code>@stellar/stellar-sdk</code> — XDR building, simulation, submission
Styling	Custom CSS — Glassmorphism dark theme, neon accents
Transaction flow	Simulate → Prepare → Sign (Freighter) → Send → Poll confirmation
Read-only calls	<code>simulateTransaction</code> via a static demo address (no wallet required for views)

Project Structure

```

stellar-pendle/
├── client/                                # React/Vite user-facing frontend
│   └── src/
│       ├── lib/
│       │   ├── stellar.js                # Low-level contract read helpers
│       │   └── (simulate only)
│       │       ├── stellar-wrapper.js    # Write transaction helpers (sign
│       │       └── + send via Freightier)
│       ├── pages/
│       │   ├── LandingPage.jsx           # Protocol introduction
│       │   ├── MarketsPage.jsx           # PT/YT market overview and rates
│       │   ├── MarketplacePage.jsx       # Buy/sell PT and YT
│       │   └── TradePage.jsx             # Mint split, redeem PT, claim
│       ├── yield, combine
│       │   ├── VaultPage.jsx             # Bridge deposit/withdraw (EVM ↔
│       │   └── Stellar)
│       ├── AdminPage.jsx                 # Dev utilities (rate adjustment,
│       └── etc.)
│           ├── components/
│           │   ├── Navbar.jsx             # Navigation
│           │   ├── TradeModal.jsx         # Trade confirmation UX
│           │   ├── TransactionModal.jsx   # Transaction status modal
│           │   └── ReturnToggle.jsx       # PT / YT return mode selector
│           └── contract_2/                # Soroban smart contracts (Rust)
│               ├── bridgeContract/        # Bridge Oracle + stsUSDe
│               └── minter/burner
│                   ├── contracts/hello-world/src/lib.rs
│                   ├── pendleWrapper/     # Derivative engine (PT/YT
│                   └── lifecycle)
│                       ├── contracts/hello-world/src/lib.rs
│                       ├── orderBook/     # P2P marketplace with auto-
│                       └── matching
│                           ├── contracts/hello-world/src/lib.rs
│                           └── token/      # Custom Soroban token utility
│                               (SAC helpers)
│                                   ├── src/
│                                   │   ├── contract.rs
│                                   │   ├── balance.rs
│                                   │   ├── allowance.rs
│                                   │   ├── admin.rs
│                                   └── metadata.rs

```

```
|      └─ storage_types.rs
|
|─ admin/
|   └─ relayer/                                # Node.js cross-chain relayer +
yield oracle                                  # Main relayer: bridge listener +
|   |   └─ relayer.js                          # Utility to bootstrap AMM
oracle pusher
|   |   └─ seed_amm.js
liquidity
|   └─ bridge-frontend/                        # Admin UI for bridge operations
(React/Vite)
|
└─ README.md
```

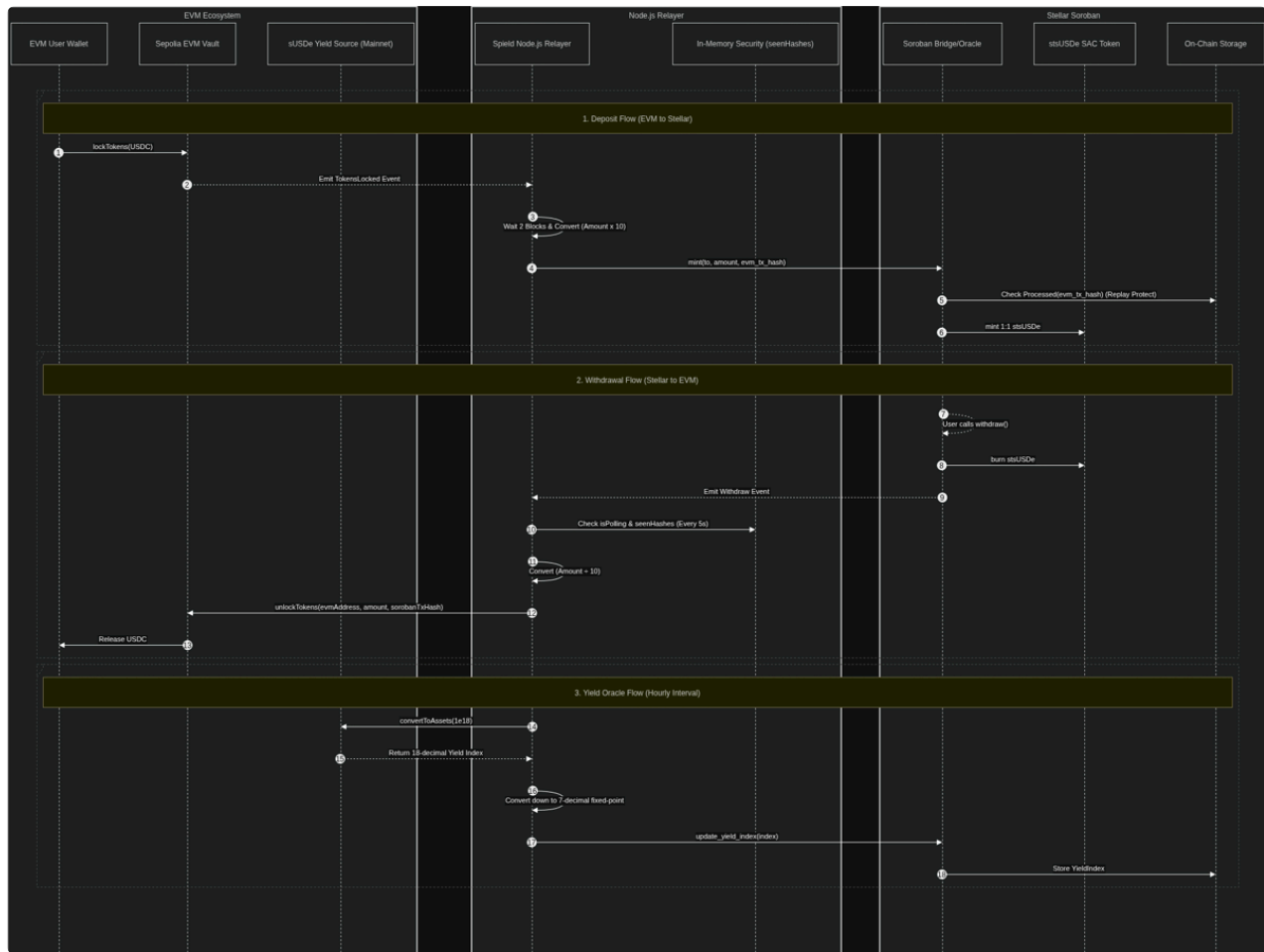

Security Considerations

Area	Risk	Current Mitigation	Future Path
Relayer Trust	Compromised relayer key could mint unbacked tokens	<code>relayer.require_auth()</code> on all privileged calls	Multi-sig or decentralized oracle network
Maturity Enforcement	Premature PT redemption	<code>ledger.timestamp() < maturity</code> — on-chain, uncircumventable	N/A
Yield Manipulation	Inflating yield index to drain vault	<code>update_yield_index</code> requires relayer auth	Decentralized oracle feeds
Re-entrancy	Draining contracts via callback loops	Soroban execution model prevents re-entrancy by design	N/A
Replay Attacks	Double-minting from same EVM tx	<code>Processed(BytesN<32>)</code> permanent storage; panics on dup	N/A
Event Double-Processing	Relayer processing same withdrawal twice	<code>isPolling</code> guard + <code>seenHashes</code> in-memory set	Persistent dedup store

Links

Resource	URL
Live App	https://spielf.vercel.app
Demo Video	https://youtu.be/VISY-C441A0
GitBook Docs	https://spielf.gitbook.io/self-docs
User Feedback	https://docs.google.com/spreadsheets/d/1fkH7NQGINKWL7JLe3ldExt4klfdEJcwlracBxBXw574

Architecture Diagram



Yield stripping model (PT + YT)

Split

You enter with a base asset (typically **USDe**). Spield converts it into the yield-bearing position (**stsUSDe**).

Against **1 stsUSDe**, Spield issues:

- **1 PT-stsUSDe**
- **1 YT-stsUSDe**

Conceptually:

 **$1 \text{ stsUSDe} \Rightarrow 1 \text{ PT-stsUSDe} + 1 \text{ YT-stsUSDe}$**

What each token represents

PT-stsUSDe (principal)

- Claim on the principal value at maturity.
- Bond-like payoff.
- Used to target a fixed return.

YT-stsUSDe (yield)

- Claim on the yield generated over the term.
- Variable payoff.
- Used to take leveraged exposure to yield.

Pricing intuition

The market prices PT and YT . Their prices add up to the underlying value.

- If yield expectations rise, **YT price** rises.
- If yield expectations fall, **PT price** rises.

Why this works

Spield does not need to predict yield. It only needs to:

- Track principal vs yield claims.
- Enforce settlement rules.
- Keep accounting consistent.

User flows

Two user profiles exist. They can also be the same person at different times.

Flow A: fixed outcome user (PT buyer)

This user wants a predictable payoff. They buy PT at a discount.

Example (illustrative):

- Buy **1 PT-stsUSDe** for **0.95 USDe**
- Redeem at maturity for **1.00 USDe**

✓ **Fixed profit = 1.00 - 0.95 = 0.05 USDe**

Implied fixed rate

If t is time-to-maturity (years):

$$1.00 = 0.95 \times (1 + r)^t$$

PT behaves like a zero-coupon bond.

Flow B: yield risk user (YT buyer)

This user wants exposure to realized yield. They pay a small upfront cost.

Example (illustrative):

- Buy **1 YT-stsUSDe** for **0.05 USDe**
- Receive the realized yield at settlement

Breakeven

 You break even when:

Realized yield per 1 stsUSDe = 0.05 USDe

Profit / loss

- Profit when realized yield is **greater** than YT price.
- Loss when realized yield is **less** than YT price.

Leverage intuition

YT controls the yield of **1.00 USDe** of principal. But it may cost **0.05 USDe**.

So the yield exposure is effectively “levered”.

$$\text{Leverage} = \frac{1.00}{0.05} = 20\times$$

Yield index and yield calculation (YT)

This page defines the **exact yield math** for YT holders. It uses your required formula.

Yield index

Spield tracks a **yield index** for the underlying stsUSDe strategy. It represents accumulated yield per 1 unit of stsUSDe.

- `pastIndex`: index value when the user acquired YT.
- `currentIndex`: latest index value at settlement time.

In these docs, the index is expressed in **USDe per 1 stsUSDe**. So $(\text{currentIndex} - \text{pastIndex})$ is the yield earned per 1 YT unit.

You can think of the index as the **stsUSDe share price** in USDe terms. If stsUSDe accrues yield, this number trends upward.

The index can be updated periodically. It must be derived from on-chain or verifiable sources.

Yield formula (required)

✓ **Yield owed to user = (currentIndex - pastIndex) × total YT bought**

Where:

- `total YT bought` is the number of **YT-stsUSDe** the user holds.
- The index difference is the yield per 1 YT.

APY calculation (from the yield index)

Use the yield index to compute an annualized rate from a 7-day change.

$$\checkmark \text{ APY} = ((\text{currentIndex} / \text{sevenDaysOldIndex})^{(365/7)}) - 1$$

Where:

- `currentIndex`: the latest index value.
- `sevenDaysOldIndex`: the index value from exactly 7 days earlier.

Example

- `sevenDaysOldIndex = 1.0000`
- `currentIndex = 1.0010`

$$\text{APY} = \left(\left(\frac{1.0010}{1.0000} \right)^{\frac{365}{7}} \right) - 1$$

This converts the 7-day growth rate into a compounded yearly rate.

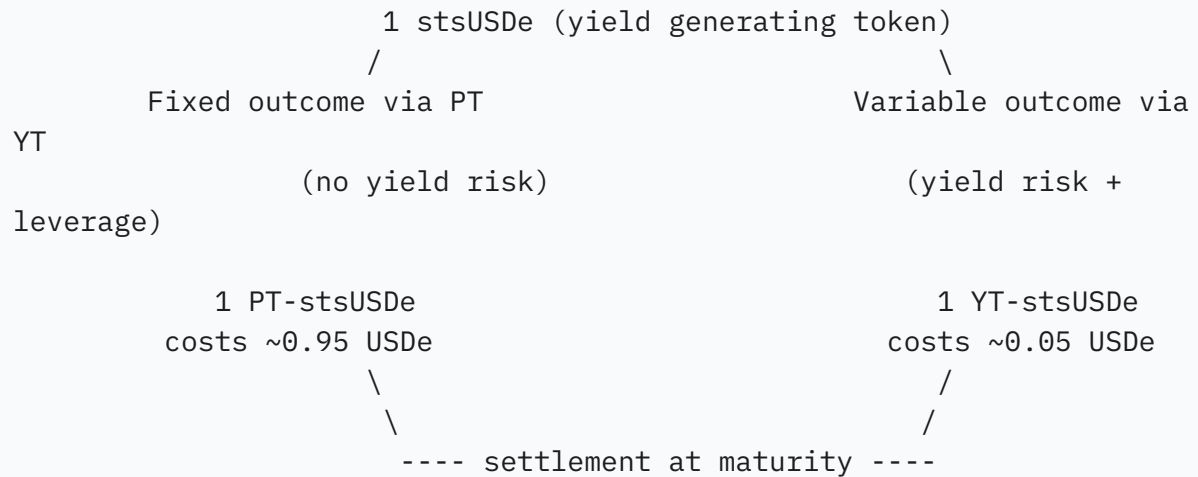
Calculation walkthrough (based on your diagram)

User perspective:

- Start with **1 stsUSDe** (yield generating token).
- Split into PT and YT.
- PT targets fixed outcome.

- YT tracks yield via the index.

Diagram-style flow



PT redemption: 1.00 USDe per PT

YT payout: $(\text{currentIndex} - \text{pastIndex}) \times (\text{total YT held})$

MVP assumption: underlying yield range over the term is ~4% to 5%.

Recomposition check (illustrative pricing):

0.95 USDe (PT price) + 0.05 USDe (YT price) = 1.00 USDe

Example

- `pastIndex = 1.0000`
- `currentIndex = 1.0450`
- `total YT bought = 300`

Yield owed:

$$(1.0450 - 1.0000) \times 300 = 13.5$$

So the user receives **13.5 USDe** of yield.

Notes

- If `currentIndex < pastIndex`, the formula returns a negative value. Treat this as a loss or clamp at 0, depending on product rules.
- If the user buys YT multiple times, track per-lot `pastIndex`.

Maturity, settlement, and capital conservation

What happens at maturity

At maturity, PT and YT settle. The system recomposes to the underlying economics.

Settlement outcomes

PT holder

- Redeems PT for **principal at face value**.
- Example: 1 PT-stsUSDe → 1.00 USDe.

YT holder

- Receives the realized yield.
- In this doc set, yield is computed by the index rule:

$$\text{Yield} = (\text{currentIndex} - \text{pastIndex}) \times \text{YT amount}$$

Total value realized

Total economic value at maturity:

$$V_{total} = 1.00 \text{ USDe} + Y_{gen}$$

Where Y_{gen} is the realized yield on the underlying.

Capital conservation (the invariant)

① Spield **does not** create or destroy capital. It only reallocates yield.

Before maturity, market pricing satisfies:

$$\text{Value}(\text{stsUSDe}) = \text{Value}(\text{PT}) + \text{Value}(\text{YT})$$

If the strategy produces yield:

- PT gets the principal outcome.
- YT gets the variable yield outcome.

The protocol stays solvent if it enforces settlement and reserves correctly.

Orderbook marketplace (PT/YT trading)

Buy and sell PT-stsUSDe and YT-stsUSDe on an orderbook before maturity.

Spield supports a secondary market where users can **buy and sell PT and YT** before maturity.

This is an **orderbook marketplace**. Traders place bids and asks. Orders match when prices cross.

What is traded

Two tokens are listed:

- **PT-stsUSDe** (principal position)
- **YT-stsUSDe** (yield position)

Typical quote assets:

- **USDe** (most intuitive for users)
- any other supported Stellar asset pair (implementation choice)

How trading changes outcomes

Trading does not change settlement rules. It changes **who holds** PT and YT at settlement.

- If you **sell PT**, you give up the fixed maturity redemption.
- If you **sell YT**, you give up future yield accrual entitlement.
- If you **buy PT**, you lock in a fixed-outcome position at your entry price.
- If you **buy YT**, you take exposure to realized yield from your entry index onward.

Pricing intuition

Orderbook pricing encodes expectations:

- **PT price** implies an annualized fixed return to maturity.
- **YT price** implies expected realized yield over the remaining term.

Across the system, the accounting identity should hold in the market:

 Before maturity, pricing is constrained by:

$$\text{Value}(\text{stsUSDe}) = \text{Value}(\text{PT-stsUSDe}) + \text{Value}(\text{YT-stsUSDe})$$

Orderbook mechanics

Keep the MVP simple:

- Limit orders.
- Matching happens on price-time priority.
- Users can enter or exit positions any time before maturity.

Risks

- Spreads and low liquidity can make entry/exit expensive.
- YT can be volatile. It is a “pure yield” bet.
- PT can trade below face value. Duration and rate risk matter.